

Session vs JWT Authentication — Complete Sequential Flow

Goal

Understand:

- What a Session is
 - What a JWT is
 - What "stateful" means
 - What "stateless" means
 - How login works
 - How profile/orders/cart requests work
 - Where cart data is stored
 - Why people say the server "remembers" users
-

Part 1: The Core Idea

When a user visits a website, HTTP is stateless by default.

Example:

Request 1:

Login

Request 2:

View Profile

Request 3:

View Orders

Request 4:

View Cart

The server receives each request independently.

Without authentication, the server cannot automatically know that all requests came from the same person.

So we need a way to identify the user across requests.

Two common approaches:

1. Session-Based Authentication (Stateful)
2. JWT-Based Authentication (Stateless)

Part 2: Session-Based Authentication (Stateful)

Login Flow

Alice sends:

```
Username: Alice
Password: 123
```

Step 1: Server verifies credentials

Server checks database:

```
Users Table

Alice
Password = 123
```

Credentials are correct.

Step 2: Server creates a Session

Server generates:

```
Session ID = S123
```

Step 3: Server stores session data

Example:

```
Session Store (Redis)

S123
↓
```

```
{  
  userId: 42,  
  username: "Alice"  
}
```

This is server-side state.

Step 4: Server sends Session ID to browser

Cookie:

```
SESSIONID=S123
```

Browser stores cookie automatically.

Important

The cookie contains:

```
SESSIONID=S123
```

The cookie does NOT contain:

```
User details  
Cart  
Orders  
Password
```

The cookie is only a key.

Think of it like a locker key.

Part 3: Viewing Profile Using Session

Browser sends:

```
GET /profile
```

```
Cookie:  
SESSIONID=S123
```

Backend Flow

Step 1:

Read cookie

```
S123
```

Step 2:

Lookup session store

```
S123  
↓  
userId = 42
```

Step 3:

Fetch user data

```
Users Database  
↓  
User 42
```

Step 4:

Return profile.

Part 4: Viewing Orders Using Session

Browser:

```
GET /orders
```

```
Cookie:  
SESSIONID=S123
```

Backend:

```
Read Cookie  
↓  
Find Session  
↓  
Get UserId  
↓  
Query Orders DB  
↓  
Return Orders
```

Part 5: Viewing Cart Using Session

There are two possibilities.

Option A: Cart Stored Inside Session

Small applications sometimes do this.

Session Store:

```
S123  
↓  
{  
  userId: 42,  
  cart: [Book]  
}
```

Request:

```
GET /cart  
Cookie: SESSIONID=S123
```

Flow:

```
Read Cookie
↓
Lookup Session
↓
cart = [Book]
↓
Return Cart
```

No separate cart database needed.

Option B: Cart Stored in Database (Most Real Systems)

Session Store:

```
S123
↓
{
  userId: 42
}
```

Cart Database:

```
UserId: 42
↓
[Book]
```

Request:

```
GET /cart
Cookie: SESSIONID=S123
```

Flow:

```
Read Cookie
↓
Lookup Session
↓
Get UserId
↓
Query Cart DB
↓
Return Cart
```

This is how large systems usually work.

Why Session Is Called Stateful

Because the server stores information.

Example:

```
S123
↓
Alice
```

The server maintains state between requests.

Server memory exists.

Without that stored session:

```
SESSIONID=S123
```

means nothing.

The server must remember:

```
S123 → Alice
```

That remembered mapping is the state.

Part 6: JWT Authentication (Stateless)

Instead of creating a server-side session, the server creates a JWT.

Login Flow

Alice sends:

```
Username: Alice  
Password: 123
```

Server verifies credentials.

Instead of creating:

```
S123
```

Server creates:

```
JWT
```

Example payload:

```
{  
  "userId": 42,  
  "name": "Alice",  
  "role": "USER",  
  "exp": 9999999999  
}
```

Server signs it cryptographically.

Server sends JWT to browser.

Browser stores it.

Important

Server does NOT store:

```
JWT123 → Alice
```

No session notebook exists.

Part 7: Viewing Profile Using JWT

Browser sends:

```
GET /profile

Authorization:
Bearer JWT
```

Backend:

```
Verify Signature
↓
Verify Expiry
↓
Extract UserId
↓
Query User DB
↓
Return Profile
```

After response:

```
Forget Request
```

No session storage.

Part 8: Viewing Orders Using JWT

Browser:

```
GET /orders

Authorization:
Bearer JWT
```

Backend:

```
Verify JWT
↓
```

```
Extract UserId
↓
Query Orders DB
↓
Return Orders
```

No session lookup.

Part 9: Viewing Cart Using JWT

Browser:

```
GET /cart

Authorization:
Bearer JWT
```

Backend:

```
Verify JWT
↓
Extract UserId
↓
Query Cart DB
↓
Return Cart
```

Cart still comes from database.

JWT does not contain the cart.

Why JWT Is Called Stateless

Because the server does not maintain authentication state.

The server never stores:

```
JWT → Alice
```

Instead:

Every request contains proof of identity.

```
JWT
↓
Verify
↓
Trust User
```

After responding:

```
Forget Request
```

The next request starts fresh.

Part 10: What Does "Remembering Previous Interactions" Actually Mean?

Many beginners misunderstand this.

It DOES NOT mean:

```
Store every request forever
```

It means:

```
Preserve user context across requests
```

Examples:

- Who the user is
 - Whether the user is logged in
 - User preferences
 - Temporary cart data
-

Session

Server remembers by storing:

```
S123
↓
Alice
```

Server owns the memory.

JWT

Server does not remember authentication state.

User carries identity proof:

```
JWT
↓
Alice
```

Client owns the identity token.

Final Comparison

Feature	Session	JWT
Authentication State Stored	Server	Client Token
Stateful?	Yes	No
Session Store Needed?	Yes	No
Cookie Contains	Session ID	Usually JWT or Access Token
Server Lookup Required?	Session Lookup	Token Verification
Scales Across Servers Easily?	Harder	Easier
Cart Stored in Cookie/Token?	No	Usually No
Cart Usually Stored In	Session or DB	DB
Server Remembers User?	Yes	No

One-Sentence Memory Trick

Session

"Give me your session ID and I'll check MY notebook to know who you are."

JWT

"Show me YOUR identity card and I'll verify it myself."